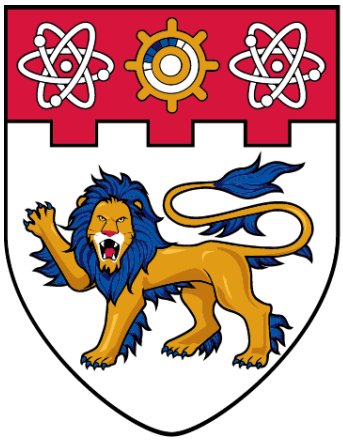

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC3000: Artificial Intelligence

Lab Assignment 2 Documentation

AY24/25 | SEM 2 | SCSB

Name	Matriculation Number
Harikrishnan Vinod	U2321114H

Question 1

Part-1

FOL Statements

competitor(sumsum, appy).
boss(stevey, appy).
developed(sumsum, galactica_s3).
smart_phone_technology(galactica_s3).
stole(stevey, galactica_s3).
∀X (smart_phone_technology(X) → business(X)).
∀X, Y (competitor(X, Y) → rival(X, Y)).
∀X, Y, Z, W (boss(X, Y) ∧ competitor(W, Y) ∧ developed(W, Z) ∧ stole(X, Z) → unethical(X)).

Part-2

Prolog Clauses

```
% Facts
competitor(sumsum, appy).
boss(stevey, appy).
developed(sumsum, galactica_s3).
smart_phone_technology(galactica_s3).
stole(stevey, galactica_s3).

% Rules
business(X) :- smart_phone_technology(X).
rival(X, Y) :- competitor(X, Y).
unethical(X) :-
    boss(X, Y),
    stole(X, Z),
    business(Z),
    competitor(W, Y),
    developed(W, Z).
```

Part-3

Question 2

Part-1

FOL Statements

∀X (offspring(queen_elizabeth, X) ∧ male(X) → priority(X, male_group)).
∀X (offspring(queen_elizabeth, X) ∧ female(X) → priority(X, female_group)).
∀X, Y (older(X, Y) ∧ same_gender(X, Y) → precedes(X, Y)).
succession_order(male_group, then female_group).
succession_line = order(males by age), then (females by age).

SWI-Prolog (AMD64, Multi-threaded, version 9.3.22)

File Edit Settings Run Debug Help

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.3.22)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit 8;;https://www.swi-prolog.orghttps://www.swi-prolog.org8;;
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- cd('C:/NTU/Y2S2/SC3000/Code/SC3000/SC3000-Assignment-2').
true.

?- ['SCSBHari_q2_1.pl'].
true.

?- trace.
true.

[trace]  ?- succession_list(Order).
   Call: (12) succession_list(_20192) ? creep
^  Call: (13) findall(_21840, (child(_21840), male(_21840)), _21856) ? creep
   Call: (18) child(_21840) ? creep
   Call: (19) offspring(queen_elizabeth, _21840) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_charles) ? creep
   Exit: (18) child(prince_charles) ? creep
   Call: (18) male(prince_charles) ? creep
   Exit: (18) male(prince_charles) ? creep
   Redo: (19) offspring(queen_elizabeth, _21840) ? creep
   Exit: (19) offspring(queen_elizabeth, princess_ann) ? creep
   Exit: (18) child(princess_ann) ? creep
   Call: (18) male(princess_ann) ? creep
   Fail: (18) male(princess_ann) ? creep
   Redo: (19) offspring(queen_elizabeth, _21840) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_andrew) ? creep
   Exit: (18) child(prince_andrew) ? creep
   Call: (18) male(prince_andrew) ? creep
   Exit: (18) male(prince_andrew) ? creep
   Redo: (19) offspring(queen_elizabeth, _21840) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_edward) ? creep
   Exit: (18) child(prince_edward) ? creep
   Call: (18) male(prince_edward) ? creep
   Exit: (18) male(prince_edward) ? creep
^  Exit: (13) findall(_21840, user:(child(_21840), male(_21840)), [prince_charles, prince_andrew, prince_edward]) ? creep
   Call: (13) order_by_birth([prince_charles, prince_andrew, prince_edward], _42648) ? creep
^  Call: (14) sort:predsort(order_by_age, [prince_charles, prince_andrew, prince_edward], _42648) ? creep
   Call: (17) order_by_age(_46432, prince_andrew, prince_edward) ? creep
   Call: (18) older(prince_andrew, prince_edward) ? creep
   Exit: (18) older(prince_andrew, prince_edward) ? creep
   Exit: (17) order_by_age(<, prince_andrew, prince_edward) ? creep
   Call: (17) order_by_age(_50062, prince_charles, prince_andrew) ? creep
   Call: (18) older(prince_charles, prince_andrew) ? creep
   Exit: (18) older(prince_charles, prince_andrew) ? creep
   Exit: (17) order_by_age(<, prince_charles, prince_andrew) ? creep
^  Exit: (14) sort:predsort(user:order_by_age, [prince_charles, prince_andrew, prince_edward], [prince_charles, prince_andrew, prince_edward]) ? creep
   Exit: (13) order_by_birth([prince_charles, prince_andrew, prince_edward], [prince_charles, prince_andrew, prince_edward]) ? creep
^  Call: (13) findall(_55518, (child(_55518), female(_55518)), _55534) ? creep
   Call: (18) child(_55518) ? creep
   Call: (19) offspring(queen_elizabeth, _55518) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_charles) ? creep
   Exit: (18) child(prince_charles) ? creep
   Call: (18) female(prince_charles) ? creep
   Fail: (18) female(prince_charles) ? creep
   Redo: (19) offspring(queen_elizabeth, _55518) ? creep
   Exit: (19) offspring(queen_elizabeth, princess_ann) ? creep
   Exit: (18) child(princess_ann) ? creep
   Call: (18) female(princess_ann) ? creep
   Exit: (18) female(princess_ann) ? creep
   Redo: (19) offspring(queen_elizabeth, _156) ? creep
   Redo: (19) offspring(queen_elizabeth, _21840) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_edward) ? creep
   Exit: (18) child(prince_edward) ? creep
   Call: (18) male(prince_edward) ? creep
   Exit: (18) male(prince_edward) ? creep
^  Exit: (13) findall(_21840, user:(child(_21840), male(_21840)), [prince_charles, prince_andrew, prince_edward]) ? creep
   Call: (13) order_by_birth([prince_charles, prince_andrew, prince_edward], _42648) ? creep
^  Call: (14) sort:predsort(order_by_age, [prince_charles, prince_andrew, prince_edward], _42648) ? creep
   Call: (17) order_by_age(_46432, prince_andrew, prince_edward) ? creep
   Call: (18) older(prince_andrew, prince_edward) ? creep
   Exit: (18) older(prince_andrew, prince_edward) ? creep
   Exit: (17) order_by_age(<, prince_andrew, prince_edward) ? creep
   Call: (17) order_by_age(_50062, prince_charles, prince_andrew) ? creep
   Call: (18) older(prince_charles, prince_andrew) ? creep
   Exit: (18) older(prince_charles, prince_andrew) ? creep
   Exit: (17) order_by_age(<, prince_charles, prince_andrew) ? creep
^  Exit: (14) sort:predsort(user:order_by_age, [prince_charles, prince_andrew, prince_edward], [prince_charles, prince_andrew, prince_edward]) ? creep
   Exit: (13) order_by_birth([prince_charles, prince_andrew, prince_edward], [prince_charles, prince_andrew, prince_edward]) ? creep
^  Call: (13) findall(_55518, (child(_55518), female(_55518)), _55534) ? creep
   Call: (18) child(_55518) ? creep
   Call: (19) offspring(queen_elizabeth, _55518) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_charles) ? creep
   Exit: (18) child(prince_charles) ? creep
   Call: (18) female(prince_charles) ? creep
   Fail: (18) female(prince_charles) ? creep
   Redo: (19) offspring(queen_elizabeth, _55518) ? creep
   Exit: (19) offspring(queen_elizabeth, princess_ann) ? creep
   Exit: (18) child(princess_ann) ? creep
   Call: (18) female(princess_ann) ? creep
   Exit: (18) female(princess_ann) ? creep
   Redo: (19) offspring(queen_elizabeth, _156) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_andrew) ? creep
   Exit: (18) child(prince_andrew) ? creep
   Call: (18) female(prince_andrew) ? creep
   Fail: (18) female(prince_andrew) ? creep
   Redo: (19) offspring(queen_elizabeth, _156) ? creep
   Exit: (19) offspring(queen_elizabeth, prince_edward) ? creep
   Exit: (18) child(prince_edward) ? creep
   Call: (18) female(prince_edward) ? creep
   Fail: (18) female(prince_edward) ? creep
^  Exit: (13) findall(_156, user:(child(_156), female(_156)), [princess_ann]) ? creep
   Call: (13) order_by_birth([princess_ann], _12348) ? creep
^  Call: (14) sort:predsort(order_by_age, [princess_ann], _12348) ? creep
^  Exit: (14) sort:predsort(user:order_by_age, [princess_ann], [princess_ann]) ? creep
   Exit: (13) order_by_birth([princess_ann], [princess_ann]) ? creep
   Call: (13) lists:append([prince_charles, prince_andrew, prince_edward], [princess_ann], _58) ? creep
   Exit: (13) lists:append([prince_charles, prince_andrew, prince_edward], [princess_ann], [prince_charles, prince_andrew, prince_edward, princess_ann]) ? creep
   Exit: (12) succession_list([prince_charles, prince_andrew, prince_edward, princess_ann]) ? creep
Order = [prince_charles, prince_andrew, prince_edward, princess_ann].

[trace]  ?-
```

Part-2

FOL Statements

∀X (offspring(queen_elizabeth, X) → priority(X)).
∀X, Y (older(X, Y) → precedes(X, Y)).
succession_line = order(all children by age).